

Advanced Energy Forecasting using LSTM (with Code Explanation)

Module 5 – Smart Energy Consumption Analysis Project

1. Importing Required Libraries

This section imports all necessary libraries for data handling, preprocessing, model building, training, and evaluation. TensorFlow/Keras is used for building the LSTM network.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.optimizers import Adam
```

NumPy and Pandas are used for numerical operations and data manipulation. MinMaxScaler normalizes energy values between 0 and 1. Sequential allows stacking neural network layers. LSTM layers learn temporal dependencies, Dropout prevents overfitting, and Adam is used as an adaptive optimizer.

2. Scaling Energy Consumption Data

Neural networks perform best when input data is normalized. Energy values are scaled using Min-Max scaling.

```
scaler = MinMaxScaler(feature_range=(0, 1))
energy_scaled = scaler.fit_transform(df[['energy']])
```

The scaler learns the minimum and maximum energy values from the training data and transforms them into a normalized range. This scaled data is used for LSTM training.

3. Creating Time Series Sequences

LSTM models require sequential input. This function creates sliding window sequences where previous time steps are used to predict the next value.

```
def create_sequences(data, time_steps=24):
    X, y = [], []
    for i in range(time_steps, len(data)):
        X.append(data[i-time_steps:i, 0])
        y.append(data[i, 0])
    return np.array(X), np.array(y)
```

For each data point, the function extracts the previous 24 energy values as input (X) and the current value as output (y). This enables the model to learn daily usage patterns.

4. Preparing Training and Testing Data

```
TIME_STEPS = 24
X, y = create_sequences(energy_scaled, TIME_STEPS)
```

```
split_index = int(0.8 * len(X))
X_train, X_test = X[:split_index], X[split_index:]
y_train, y_test = y[:split_index], y[split_index:]
```

The dataset is split chronologically into training and testing sets. Shuffling is avoided to preserve temporal order.

5. Reshaping Data for LSTM Input

```
X_train = X_train.reshape((X_train.shape[0], X_train.shape[1], 1))
X_test = X_test.reshape((X_test.shape[0], X_test.shape[1], 1))
```

LSTM expects 3D input: number of samples, time steps, and number of features. Since energy is a univariate time series, the feature dimension is set to 1.

6. Building the LSTM Model

```
model = Sequential()

model.add(LSTM(64, return_sequences=True, input_shape=(TIME_STEPS, 1)))
model.add(Dropout(0.2))

model.add(LSTM(32))
model.add(Dropout(0.2))

model.add(Dense(1))
```

The first LSTM layer captures long-term dependencies and returns sequences for the next LSTM layer. Dropout layers reduce overfitting. The final Dense layer outputs a single energy prediction.

7. Compiling and Training the Model

```
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='mean_squared_error'
)

early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.1,
    callbacks=[early_stop],
    verbose=1
)
```

The model is trained using Mean Squared Error loss. Early stopping halts training when validation performance stops improving, ensuring better generalization.

8. Model Prediction and Evaluation

```
y_pred_scaled = model.predict(X_test)

y_test_actual = scaler.inverse_transform(y_test.reshape(-1, 1))
y_pred_actual = scaler.inverse_transform(y_pred_scaled)

mae = mean_absolute_error(y_test_actual, y_pred_actual)
rmse = np.sqrt(mean_squared_error(y_test_actual, y_pred_actual))
```

```
print("LSTM MAE:", mae)
print("LSTM RMSE:", rmse)
```

Predictions are inverse-transformed to original scale before evaluation. MAE and RMSE are used to compare LSTM performance with the baseline Linear Regression model.

9. Saving the Trained Model

```
model.save("lstm_energy_model.h5")
```

The trained model is saved in HDF5 format, enabling future loading and deployment within a Flask-based application.